

Question 1

The following C function takes a simply-linked list as input argument. It modifies the list by moving the last element to the front of the list and returns the modified list. Some part of the code is left blank.

```
typedef struct node
{
    int value;
    struct node *next;
}Node;

Node *move_to_front(Node *head)
{
    Node *p, *q;
    if ((head == NULL) || (head->next == NULL))
        return head;
    q = NULL; p = head;
    while (p->next != NULL)
    {
        q = p;
        p = p->next;
    }
    _____
    return head;
}
```

Choose the correct alternative to replace the blank line.

- ☐ A q = NULL; p->next = head; head = p;
- ☐ B q->next = NULL; head = p; p->next = head;
- ☐ C head = p; p->next = q; q->next = NULL;
- ☐ D q->next = NULL; p->next = head; head = p;

Question 2

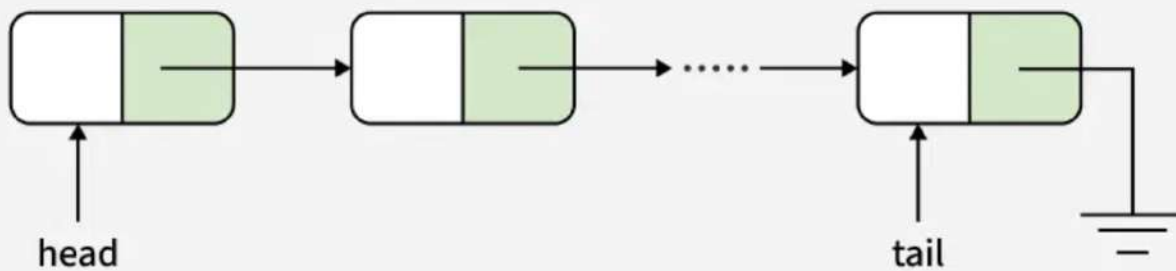
The following C function takes a single-linked list of integers as a parameter and rearranges the elements of the list. The function is called with the list containing the integers 1, 2, 3, 4, 5, 6, 7 in the given order. What will be the contents of the list after the function completes execution?

```
struct node
{
    int value;
    struct node *next;
};
void rearrange(struct node *list)
{
    struct node *p, *q;
    int temp;
    if ((!list) || !list->next)
        return;
    p = list;
    q = list->next;
    while(q)
    {
        temp = p->value;
        p->value = q->value;
        q->value = temp;
        p = q->next;
        q = p?p->next:0;
    }
}
```

- ☐ A 1,2,3,4,5,6,7
- ☐ B 2,1,4,3,6,5,7
- ☐ C 1,3,2,5,4,7,6
- ☐ D 2,3,4,5,6,7,1

Question 3

A queue is implemented using a non-circular singly linked list. The queue has a head pointer and a tail pointer, as shown in the figure. Let n denote the number of nodes in the queue. Let 'enqueue' be implemented by inserting a new node at the head, and 'dequeue' be implemented by deletion of a node from the tail.



Which one of the following is the time complexity of the most time-efficient implementation of 'enqueue' and 'dequeue', respectively, for this data structure?

- (A) $\Theta(1), \Theta(1)$
- (B) $\Theta(1), \Theta(n)$
- (C) $\Theta(n), \Theta(1)$
- (D) $\Theta(n), \Theta(n)$

Question 4

N items are stored in a sorted doubly linked list. For a delete operation, a pointer is provided to the record to be deleted. For a decrease-key operation, a pointer is provided to the record on which the operation is to be performed. An algorithm performs the following operations on the list in this order: $\Theta(N)$ delete, $O(\log N)$ insert, $O(\log N)$ find, and $\Theta(N)$ decrease-key. What is the time complexity of all these operations put together?

- ☐ A $O(\log^2 N)$
- ☐ B $O(N)$
- ☐ C $O(N \log N)$
- ☐ D $\Theta(N^2 \log N)$

Question 5

Let $SLLdel$ be a function that deletes a node in a singly linked list given a pointer to a node and a pointer to a head of the list. Similarly, let $DLLdel$ be another function that deletes a node in a doubly linked list given a pointer to the node and a pointer to the head of the list. Let n denote the number of nodes in each of the linked lists. Which one of the following choices is TRUE about the worst-case time complexity of $SLLdel$ and $DLLdel$?

- ☐ A $SLLdel$ is $O(1)$ and DLL is $O(n)$
- ☐ B Both $SLLdel$ and DLL is $O(\log(n))$
- ☐ C Both SLL and DLL are $O(n)$
- ☐ D SLL is $O(n)$ and DLL is $O(1)$

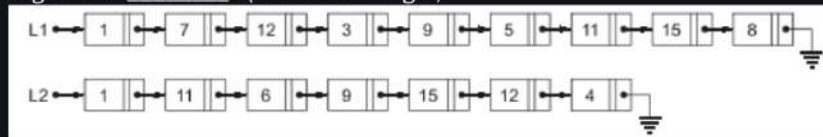
Question 6

Let LIST be a datatype for an implementation of linked list defined as follows:

```
typedef struct list {  
    int data;  
    struct list *next;  
} LIST;
```

Suppose a program has created two linked lists, L1 and L2, whose contents are given in the figure below (code for creating L1 and L2 is not provided here). L1 contains 9 nodes, and L2 contains 7 nodes.

Consider the following C program segment that modifies the list L1. The number of nodes that will be there in L1 after the execution of the code segment is _____. (Answer in integer)



Figure

```
int find(int query, LIST *list) {  
    while (list != NULL) {  
        if (list->data == query) return 1;  
        list = list->next;  
    }  
    return 0;  
}  
  
int main() {  
    ... ..  
    ptr1 = L1; ptr2 = L2;  
  
    while (ptr1->next != NULL) {  
        query = ptr1->next->data;  
        if (find(query, L2))  
            ptr1->next = ptr1->next->next;  
        else  
            ptr1 = ptr1->next;  
    }  
  
    ... ..  
    return 0;  
}
```